Лабораторная работа №1

Выявление аномалий и инцидентов ИБ в системных журналах

Инструменты: `Python`, библиотека `re` (регулярные выражения)

Рекомендуемые дополнительные: `argparse, collections, datetime`

Библиотека `re`

Библиотека `re` в `Python` — это стандартный модуль для работы с регулярными выражениями. Она позволяет искать, извлекать, заменять и разделять текст по сложным шаблонам.

Основные функции:

`re.search(pattern, string)` — сканирует строку и возвращает объект первого найденного совпадения `(Match object)`. Если совпадений нет, возвращает `None`.

`re.match(pattern, string)` — проверяет совпадение только в самом начале строки. В `Python 3.15` эта функция помечена как устаревшая в пользу `re.prefixmatch()`.

`re.fullmatch(pattern, string)` — проверяет, соответствует ли *вся* строка шаблону целиком.

`re.findall(pattern, string)` — возвращает список всех непересекающихся совпадений. Если в шаблоне есть группы захвата, вернет список кортежей.

`re.finditer(pattern, string)` — возвращает итератор, который выдает объекты `Match` для каждого совпадения. Полезен, когда нужно узнать позицию (индексы) совпадения через методы `.start() и .end()`.

`re.sub(pattern, repl, string)` — заменяет все совпадения на указанную строку или результат функции. Несопоставленные группы захвата теперь заменяются пустой строкой.

re.split(pattern, string) — разбивает строку по вхождениям шаблона. Если в шаблоне есть захватывающие скобки, текст этих групп также попадет в результирующий список.

re.compile(pattern, flags=0) — компилирует регулярное выражение в объект. Это более эффективно, если один и тот же шаблон используется многократно.

Сырые строки (Raw strings): Шаблоны почти всегда пишутся с префиксом r (например, r"\d+"). Это предотвращает двойную интерпретацию обратного слэша (\) самой строкой Python. Без r для поиска цифры пришлось бы писать "\\d+".

Флаги (Flags): Изменяют поведение поиска. Передаются в функции или в re.compile():

re.IGNORECASE (или re.I) — поиск без учета регистра.
re.MULTILINE (или re.M) — символы ^ и $ будут соответствовать началу и концу каждой строки, а не только всей строки целиком.
re.DOTALL (или re.S) — метасимвол точки . будет соответствовать любому символу, включая символ новой строки.
re.VERBOSE (или re.X) — позволяет писать шаблон в несколько строк, добавляя комментарии и пробелы для читаемости.
re.ASCII (или re.A) — заставляет спецсимволы (\w, \d, \b) работать только с ASCII-символами, отключая полную поддержку Unicode.

Пример:

```
import re

text = "Заказ 123: оплачено, Заказ 456: в обработке, Заказ 789:
отменен"

# Найти все номера заказов
numbers = re.findall(r"Заказ (\d+)", text)
# Результат: ['123', '456', '789']

# Заменить статусы на единый формат
new_text = re.sub(r": (\w+)", r": [ОБРАБОТКА]", text)
# Результат: 'Заказ 123: [ОБРАБОТКА], Заказ 456: [ОБРАБОТКА],
Заказ 789: [ОБРАБОТКА]'

# Получить позиции слов, заканчивающихся на 'о'
for match in re.finditer(r"\w+о\b", text):
    print(f"Слово '{match.group()}' с {match.start()} по
{match.end()-1} символ")
```

*Для отладки сложных выражений удобно использовать онлайн-инструменты, такие как Regex101, где можно протестировать шаблон и получить его пошаговое объяснение. (*regex101: Online Regex Tester, Builder, Converter & Debugger *)*

Задание

1. Предварительно изучить и проанализировать 10 тестовых файлов логов. Определить, в каких из них содержится подозрительная активность.
2. Написать функцию для чтения текстового файла лога. Реализовать обработку исключений на случай отсутствия файла или проблем с кодировкой.
3. Используя регулярные выражения (библиотека `re`), реализовать поиск строк, содержащих неудачные попытки входа. (`FAILED_LOGIN_PATTERN` и `INVALID_USER_PATTERN`)

4. Извлечь IP-адрес источника атаки из каждой найденной строки и сохранить его в список или словарь.

5. Добавить в скрипт аргумент командной строки (например с использованием библиотеки `argparse`) для передачи пути к входному файлу:

`python ids_lab1_StudentID.py my_log.txt.`

6. Реализовать подсчет количества неудачных попыток для каждого уникального IP-адреса (рек. использовать `collections.Counter` или `defaultdict`).

7. Ввести пороговое значение (`Threshold`). Значение по умолчанию 5 попыток. Сделать этот порог настраиваемым через аргумент командной строки (`--threshold`).

8. Реализовать алгоритм сравнения: если количество попыток с одного IP превышает порог, выводить в консоль предупреждение в формате:

`[ALERT] Brute-force detected from <IP>. Attempts: <COUNT>.`

Так же по итогам работы программы должна формироваться общая статистика

`Статистика активности IP`

`Пример выводимых данных:`

```
[ALERT] Brute-force/IP Spraying detected from 203.0.113.45. Attempts: 7
[ALERT] Brute-force/IP Spraying detected from 192.0.2.1. Attempts: 10
[ALERT] Brute-force/IP Spraying detected from 10.0.0.5. Attempts: 9

Статистика активности IP (все время в файле):
  2026-10-13 11:09:50.320225: 38 попыток
```

9. [Дополнительное задание] Добавить аргумент `--window` (минуты). Реализовать логику, при которой скрипт анализирует только те события, которые произошли в течение последних N минут *относительно времени последней записи в файле*. Для этого потребуется извлечь временную метку из строки лога и преобразовать её в объект `datetime`.

10. Протестировать разработанную программу на тестовом наборе логов. Сформировать таблицу результатов тестирования.

(Имя файла | Наличие алерта | Ожидаемый результат | Фактический результат).